From Attack Behavior to Control-Point Attribution: A Stage-Aware Framework for Evaluating APT Defense Capability

Anonymous Author(s)

Abstract

CCS Concepts

• **Computing methodologies** → **Multi-agent systems**; • **Software and its engineering** → *Software creation and management*.

Keywords

APT defense evaluation, control-point attribution, stage-aware scoring, security benchmark

ACM Reference Format:

Anonymous Author(s). 2026. From Attack Behavior to Control-Point Attribution: A Stage-Aware Framework for Evaluating APT Defense Capability. In *Proceedings of THE ACM WEB CONFERENCE 2026 (Conference acronym 'WWW)*. ACM, New York, NY, USA, 13 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Advanced Persistent Threat (APT) campaigns unfold as multi-stage operations that may span days or months before an objective is reached. Recent industry reporting places the global median dwell time at 14 days in 2025, with cyber-espionage intrusions found considerably later[13], attributes a large share of intrusions to credential abuse and vulnerability exploitation[51], and records substantial business disruption once a breach occurs[18]. Such figures do not convey the shape of the defense. Individual actions or stages may have been detected or disrupted even when the campaign ultimately succeeded, and a campaign that failed may have been stopped by a single control acting late. Recording only whether a campaign succeeded therefore hides where defensive capability held and where it was insufficient.

Organizations respond by deploying layered stacks, but deployment is not capability. The components of such a stack emit evidence with different meanings: one removes a precondition so that an action cannot be attempted, another interrupts the action while it runs, a third records it without preventing it, and a fourth does not observe it. Counting products, or counting alerts, flattens these distinctions. An evaluation useful to a defender must instead be comparable across heterogeneous stacks, aware of where in a campaign an outcome occurred, and traceable from an aggregate figure back to a control that can be repaired.

Existing approaches supply parts of this picture. ATT&CK provides a shared vocabulary for adversary behavior[31] and public product evaluations report responses step by step[34], yet technique coverage has been shown empirically to be a weak proxy for defensive capability[52]. Provenance-based detection and attack reconstruction recover the causal origin of observed malicious activity[15, 20], and control catalogs relate adversary techniques to countermeasures[21, 33]. These outputs characterize attacker behavior, causal evidence, or candidate countermeasures, but they are not designed to provide the stable victim-side control attribution required by our repair-oriented scoring task.

A second line represents how an intrusion advances. Attack graphs and reachability models reason about attack paths and their composition[46, 55], while severity and system-level risk models quantify exposure along a different axis[10, 28]; step-level results have also been re-linked into causal chains to recover campaign structure[44]. Campaign progress and the control-point domain associated with an action nevertheless remain orthogonal dimensions: the same control domain can be implicated at multiple phases, while actions within one phase can map to different control domains. Collapsing them onto a single axis makes a score harder to act on, not easier.

A third line aggregates observations into an overall figure. Security-metric and evaluation-validity work reports coverage, detection rates, and paired effectiveness quantities[17, 26, 59], and cautions that aggregate security numbers are sensitive to how they are constructed[49, 50]. A uniform average treats action outcomes as interchangeable. Actions within one stage are not: sometimes any one of them suffices for the attacker, sometimes all are required, and sometimes they form a loose set. An average cannot represent these alternative, conjunctive, and clustered stage semantics, and it obscures which actions and stages account for the resulting score loss.

The three limitations above point to a common missing object: an evaluation unit that keeps attacker behavior, a benchmark-side repair reference, and the evaluated system's response aligned without conflating them. We therefore pair an in-scope benchmark action with a reference primary control-point attribution fixed during benchmark construction, and with the outcome observed when a defense system under test is exercised against that action. The attribution names one victim-side control point – a mechanism, configuration, policy, or operational process – and is a benchmark-side reference, assigned under a documented procedure before any system is evaluated. The observed outcome, by contrast, records what the evaluated system did when the action was exercised. This separation matters because an attacker-side technique label does not by itself determine a victim-side repair target: actions sharing the same technique label may require different control-point attributions depending on their context. Attacker-side and victim-side attribution answer different questions, and a framework aimed at repair must preserve the second without discarding the first.

Conference acronym 'WWW, Dubai, United Arab Emirates

ACM ISBN 978-1-4503-XXXX-X/18/06
https://doi.org/XXXXXXX.XXXXXXX

Building an evaluation on this unit raises three methodological challenges. First, the attribution must remain stable across heterogeneous actions while staying repair-relevant: a schema too coarse cannot separate distinct repairs, and one too fine cannot be applied consistently. Second, campaign phase and control-point domain must be carried as separate dimensions, together with the confidence of each assignment, so that a downstream figure can be interpreted rather than merely read. Third, action-level outcomes must be aggregated under the semantics that actually hold inside a stage, and the aggregate must remain decomposable afterwards.

We address these with a three-step framework. The first step constructs the reference attribution for benchmark actions, recording a primary control-point reference together with its confidence under the benchmark schema. Secondary annotations, when present, remain contextual metadata rather than additional primary scoring targets. The second step enriches the attributed action into a scoring-ready record, adding the campaign phase at which the action occurs, the defense product categories expected to address the attributed control point, and the aggregation semantics governing its stage; phase is carried independently of the attribution, so that where an action occurs in the campaign remains separate from which control-point domain it is associated with. The third step aligns observations from a system under test to benchmark actions, normalizes them onto a common outcome vocabulary of `PREVENTED`, `BLOCKED`, `DETECTED`, and `MISSED`, and aggregates actions to stages and stages to playbooks while retaining breakdowns by phase, control-point domain, and product category. The current benchmark instance draws on 53 APT playbooks and 1,796 authoritative raw action occurrences; the benchmark presently defines 1,773 active scoring opportunities, and the control-point schema presently comprises nine top-level domains and 97 active leaves.

The main contributions of this paper are as follows:

- **An evaluation representation for repair-oriented APT defense scoring.** We define the evaluation record so that it extends an attacker-side action description by binding to it a reference primary control-point attribution and an observed defense outcome, keeping the benchmark-side attribution separate from the behavior of the system under test. The attack action remains part of the representation; what the representation adds is a repair-oriented victim-side dimension alongside the observed result.
- **A scoring-ready, stage-aware benchmark representation.** We describe a construction process that enriches each attributed action with its campaign phase, the expected defense product categories, confidence, and explicit stage aggregation semantics, keeping campaign progress and victim-side control attribution as separate dimensions of one record.
- **A stage-aware aggregation and reporting model.** We separate an action's outcome from the weight of the stage containing it, aggregate outcomes under each stage's recorded OR, AND, or Cluster semantics rather than a uniform average, compose action to stage to playbook scores, and preserve decomposition by action, control point, and phase.

2 Related Work

Work relevant to this paper answers four questions using four kinds of evidence. *Prospective structural evaluation* asks what an attacker could achieve given a model of configurations, vulnerabilities, and reachability. *Empirical replay observation* asks what a deployed defense did when specific actions were executed against it. *Defense-effectiveness scoring* asks how such results combine into a comparable quantity. *Repair attribution* asks which control should be fixed as a consequence. The four are not interchangeable: a reachable path, an alert, a blocking decision, and a remediation target are different objects, and a score over one cannot be read as a statement about another. We therefore organize prior work by the question it answers and by its unit of analysis.

2.1 Attack Knowledge Bases, Adversary Emulation, and Defense Benchmark Construction

ATT&CK supplies the tactic and technique vocabulary used to normalize adversary behavior and derive canonical phases in our benchmark [31]. ATT&CK Evaluations record how individual products expose or detect procedures during emulation [34], while tooling such as CALDERA [30] makes technique-level tests executable and harnesses such as DELTA orchestrate attacks against a target [22]. These resources yield executable actions and per-product observations, but do not specify how such observations become comparable outcomes, nor how an aggregate points back to a control worth repairing.

Two lines of work evaluate a deployed defense at action granularity by injection or replay, and they are the closest prior art to our evaluation pipeline. SAIBERSOC injects synthetic ATT&CK-derived attacks into a live security operations center [42]. Its evaluated object is the operational center as a whole, monitoring infrastructure together with the analysts who staff it; its unit of analysis is an injected attack instance; and it reports detection accuracy separated into identification and investigation, together with time-to-investigation, compared between differently configured centers. Cao et al. replay real-world attack variants against several detection methods and report detection capability per variant [4]; their evaluated object is the detector and their unit is the replayed variant. Both pipelines coincide with ours up to the moment an outcome is recorded. They diverge in what that outcome is designed to explain: SAIBERSOC explains variance attributable to center configuration and analyst process, and Cao et al. explain per-variant detection capability, each appropriate to its own evaluation objective. Our evaluation record instead carries four elements together, the benchmark action, a reference primary control-point attribution fixed before evaluation, the observed defense outcome, and the action's campaign-stage context, so that an aggregate can be decomposed along any of them.

Automated workflows spanning defense generation, enforcement, and evaluation occupy adjacent ground. CD-GEE covers these phases and validates on case-study networks [9]. Its reported evaluation unit is a difference in security posture before and after defense deployment, whereas ours is an outcome observed for a specific evaluated benchmark action. The comparison therefore

operates at different granularities: system-level posture in CD-GEE and action-level defense evidence in our framework.

Mappings between controls and attacker behavior are maintained both in research and in industry. D3FEND organizes countermeasures and their relationships to adversary behavior [21, 32], prospectively identifying which defensive techniques may be relevant to a behavior, and the MITRE Center for Threat-Informed Defense publishes thousands of technique-to-control mappings with a documented methodology [33]. Both organize catalogs of defensive controls. Our framework instead uses a victim-side control-point taxonomy and a documented crosswalk between attributed control points and primary or compensating defense categories, and reports agreement statistics for a sampled annotation pilot. Vendor-specific practitioner workflows provide further context: breach-and-attack-simulation and adversarial-exposure-validation products execute attack procedures against a live environment and report control gaps to their operators. We note the practice for its shared concern with observed rather than assumed coverage, and draw no comparison with it, since its methods and results are documented per product.

Our scoring layer consumes observations from systems never designed to be scored. Provenance-based intrusion detection has been systematized as a field [19]; representatively, HOLMES correlates suspicious flows into campaign-level scenarios [29] and KAIROS reconstructs compact attack summaries from whole-system provenance [6]. On the enforcement side, SANE renders some connectivity architecturally impossible [5], FIREMAN yields static policy findings [58], and programmable-data-plane defenses interrupt traffic during execution [57, 60]. These are representatives of evidence types rather than an inventory: what matters for scoring is the distinction between architectural impossibility, runtime interruption, observation without interruption, and absence of evidence, which map to `PREVENTED`, `BLOCKED`, `DETECTED`, and `MISSED`.

2.2 ATT&CK-Based Coverage and Evaluation Reinterpretation

A coverage count over ATT&CK techniques is a tempting capability metric, and it has already been tested. Virkud et al. examine how endpoint detection products and public rulesets use ATT&CK and show empirically that technique coverage is a poor indicator of defensive capability [52]. Their result is diagnostic, and we build on it rather than re-deriving it. We therefore do not use technique coverage as the score. Evaluation is operationalized one level down, at an attributed action: attributed benchmark actions carry a reference primary control-point attribution, and it is the observed outcome on such an action, rather than the presence of a technique in a coverage list, that enters the score.

Step-level results have also been aggregated across an attack chain. Shen et al. re-link the steps of a published ATT&CK Evaluations round into causal attack graphs, motivated by the observation that published reports present steps in isolation [44]. The inputs differ in direction: their evidence is the released output of one existing evaluation, scoped to endpoint products, while ours is a benchmark and attribution layer built from playbooks, which allows a previously unevaluated cross-category defense stack to be scored and its score loss traced to a control.

Both depend on the provenance of the mappings that produce them. Threat-intelligence extraction [25] and temporal-relation learning over attack steps [45] show such mappings to be derived artifacts, and operational feeds differ substantially in coverage and timeliness [24]. We therefore record a confidence label with each attributed action, and the coding guide used during annotation distinguishes how an attribution was reached, rather than treating all mappings as equivalent evidence.

2.3 Defense Capability and Effectiveness Scoring

Comparable defense evaluation has been unified before, on a different denominator. Iannacone and Bridges survey the area and express intrusion-detection metrics, cyber-exercise scoring, and security economics through a common cost term [17]. Ours is a stage-weighted interception value paired with a repairable control target, so the output is a score that localizes rather than a cost that totals. The cost-oriented lineage is longer still [3, 14], and it illustrates a failure mode we avoid: collapsing incomparable consequences into one index produces a number that is hard to act on, which is why we also report by phase, control-point domain, and product category.

DEFIA is particularly close in intent, scoring the utility of attack behaviors prevented and recording the point at which a defense acts [26]. Its unit of analysis is an attack behavior and its semantics are effectiveness-oriented, resting on whether a behavior was prevented and on where the defense took effect. Three properties of our unit separate the two. Observed outcomes occupy a four-level normalized scale, `PREVENTED`, `BLOCKED`, `DETECTED`, and `MISSED`, rather than a prevented/not-prevented distinction. Attributed benchmark actions carry a reference primary control-point attribution assigned during benchmark construction, independently of the system under test. And stage membership carries explicit alternative, conjunctive, and clustered composition, which our benchmark must retain when aggregating observed action outcomes from action to stage to playbook.

Prior work on moving-target-defense evaluation already separates complementary views of defense effectiveness. Zaffarano et al. define a quantitative effectiveness framework in which mission-side and attack-side outcomes are measured separately rather than folded into a single figure [59], and Taylor et al. apply that framework under automated experimentation, reporting the mission and attack metrics as a pair [48]. This separation is the closest structural precedent for our coverage-versus-hit-rate reporting. The evaluated object differs: theirs is a single defense class, moving-target mechanisms, whereas ours is a heterogeneous multi-product stack, so the pairing becomes coverage of the action set structurally applicable to a product category, measured against the hit rate observed within that coverage. Neither work attaches a victim-side control reference to the action or composes outcomes under within-stage semantics. Detection outcomes have likewise been weighted by time: Garcia et al. apply a correcting function per window so that earlier detection scores higher [12]. The weighted quantity differs, elapsed detection time in their case and semantic attack phase combined with stage dependency in ours. Time-sensitive and multi-dimensional

weighted scoring also appears in live exercises [8, 56], where the measured object is a participating team.

Dong et al. assess provenance-based endpoint detection and response tools against operational decision factors such as client overhead and triage cost [7]. Those dimensions measure the cost of operating a tool; ours characterize observed defense outcomes in campaign context while retaining benchmark-side control-point attribution.

2.4 Multi-Stage Evaluation, Attack-Path Reasoning, and Repair Prioritization

Structural methods evaluate multi-stage risk without observing a defense in operation. CVSS scores the severity of an individual vulnerability [11, 28], and attack-graph methods reason over configurations, privileges, and reachability through symbolic model checking [46], logical derivation [38], or generation at larger scale [37]. Derived metrics extend the same view: k-zero-day safety counts the unknown vulnerabilities an attacker would require [53], network diversity estimates resilience from software heterogeneity [61], and mean time-to-compromise estimates attacker progress across protection zones [23]. Each is computed from a model of what is reachable rather than from what a deployed defense did. Attack-graph-based security measurement composes such models into overall metrics for a networked system [55], and system-level assessment aggregates risk across an organization's assets [10]. These lines organize progression, feasibility, and system-level risk, and they are well suited to those questions. The dimension they organize is nonetheless not the one our attribution supplies: where an action sits in a campaign, and how far an attacker could progress, is a separate question from which class of victim-side control the benchmark associates with that action. We therefore keep campaign phase and control-point domain as independent fields rather than deriving either from the other. Our crosswalk uses structural information in the same prospective spirit, to decide which product categories are expected to cover an action, and reports that applicability separately from empirical performance.

Attack steps have been assigned differentiated defensive value in this setting. GPLADD models multi-step attacker-defender interaction with explicit per-step timing and value [39], and later work applies the approach to defender policy evaluation and resource allocation using ATT&CK Evaluations data [40]. The mode of inference separates the two: that work optimizes allocation probabilistically under uncertainty and outputs a policy, whereas ours computes a deterministic score from observed outcomes and outputs a figure reducible to the attributed actions that account for each score loss.

Repair targets have long been selected from attack models, and this is the prior-work line closest to our attribution contribution. Noel et al. reason over an exploit-dependency graph to identify the initial conditions whose removal makes critical resources unreachable, returning a hardening set rather than a per-event judgement [35]; Wang, Noel and Jajodia develop the minimum-cost formulation of the same problem, selecting a remediation configuration under cost constraints [54]. Both operate prospectively over a system and attack model, and both output a set chosen to be sufficient for a reachability goal. The overlap is real: each produces a remediation target from an attack representation. The difference lies in when the target is fixed and what it is attached to. The framework records benchmark-side primary control-point references during benchmark construction, before any system is evaluated, and subsequently aligns observed outcomes to the corresponding benchmark records, aggregating them while action, stage, and control-point traceability is preserved. Unlike minimum-cost hardening, it does not solve for an optimal or sufficient remediation set. Neither output subsumes the other.

The word *attribution* requires disambiguation, because it already denotes a different object in this literature. ORTHRUS defines quality of attribution as the analyst effort needed to recover an attack's causal origin from detection output [20], and R-CAID embeds root-cause analysis within provenance-based detection [15]. Both reconstruct, explain, or localize attacker-side causal origins in provenance and event space, and both are evaluated on how well that reconstruction succeeds. The attribution in this paper is a different object on a different substrate: a benchmark-side victim-side control-point reference attached to a benchmark action, used for defense scoring and repair-oriented decomposition rather than for explaining an observed intrusion. Neither output subsumes the other, and the shared word should not be read as a shared task.

2.5 Evaluation Validity, Reproducibility, and Operational Evidence

A unified layer over heterogeneous systems is itself a known construction. Bilot et al. build one for provenance-based intrusion detection, ingesting multiple implementations and reporting comparable metrics [2]. The evaluated object differs: detection quality of research prototypes on public datasets in their case, a defense stack evaluated against benchmark playbook actions in our framework, where attributed benchmark records carry a reference primary control-point attribution.

Several results constrain how our scores may be interpreted. Sommer and Paxson argue that designing the evaluation is harder than building the detector [47], and the base-rate fallacy shows that a low false-positive rate can coexist with a high false-alarm rate under class imbalance [1], which is why detection denominators must be kept apart. We apply that separation to product-category scoring, reporting coverage over the structurally applicable action set apart from hit rate within it. Verendel finds that quantified-security methods have generally lacked repeated testing [50], and Herley and van Oorschot argue that security claims require explicit assumptions, evidence, and limits of inference [16]. We follow these constraints by stating the evidence basis of each reported quantity.

Benchmark validity should be assessed rather than assumed. Van der Kouwe et al. catalog recurring flaws in systems benchmarking and motivate treating benchmark-validity limitations as an explicit subject rather than an implicit one [49], while Olszewski et al. show that reproduction claims depend on environment and implementation assumptions being made explicit [36]. Both concerns apply to a benchmark of this kind and shape how its reported quantities should be qualified.

One scope limitation follows. The attribution schema covers endpoint, identity, data, network, and operational controls, so a network-based testbed can validate only a replayable slice of the

Table 1: Positioning by evidence, unit of analysis, output, and remediation traceability. Rows group research lines by the question each answers; the grouping does not indicate competitor class.

Research line	Primary evidence	Unit of analysis	Typical output	Remediation traceability
Knowledge bases, emulation, and replay	Technique descriptions and emulation or replay telemetry	Technique, emulated step, or injected attack	Vocabulary, executable test, or per-attack observation	Partial; process- or capability-oriented
ATT&CK coverage and reinterpretation	Published evaluation outputs and detection rulesets	Technique or evaluation step	Coverage assessment or relinked attack graph	Indirect; scoped to one product class
Effectiveness scoring	Costs, mission metrics, or prevented-behavior utility	Cost-accruing event or attack behavior	Comparable effectiveness or cost index	Aggregate; not tied to one control
Multi-stage reasoning and repair prioritization	Configuration, topology, privileges, and modelled paths	Vulnerability, condition, or attack path	Severity, reachability, or minimal hardening set	Structural and prospective
Validity and reproducibility	Published benchmarks and evaluation designs	Benchmark, metric, or replication attempt	Methodological constraint or checklist	Not applicable
Industry mappings and practitioner validation	Control catalogs and executed attack procedures	Technique–control pair or executed procedure	Mapping set or vendor gap report	Control-oriented; documented per product
This work	Playbook action and aligned defense observation	Action–outcome–control record	Stage-aware, multi-dimensional defense score	Explicit reference primary control-point attribution

schema rather than its full scope. Actions requiring unsupported endpoint state should be treated as out of scope or as explicit preconditions rather than being assigned a defense outcome.

2.6 Synthesis and Positioning

Table 1 summarizes these lines by evidence, unit of analysis, output, and remediation traceability. Existing work separately contributes attacker and action vocabularies together with replay evidence, defense effectiveness and capability metrics, multi-stage and attack-path reasoning, and repair-prioritization or causal-attribution outputs. Each of these components is established, and we claim none of them in isolation. What this framework combines is a benchmark-side reference primary control-point attribution, an observed action outcome on a four-level normalized scale, and explicit within-stage composition semantics, held together so that a comparable defense score remains traceable to the attributed actions and control points that account for it. Simply composing the existing outputs would not achieve this, because an alert, a blocking decision, a posture difference, and a prospective path-risk value do not identify the same evaluated event and cannot be aggregated without an explicit alignment rule. That alignment rule, and the traceability it preserves, is what the action–outcome–control record provides.

3 Background and Problem Statement

This section defines the evaluation setting considered in this paper. The goal is not to provide a general introduction to APTs, but to clarify the objects that are scored by our framework: APT playbook actions, defense observations, attack stages, and reference primary control-point attributions.

3.1 APT Playbooks, Actions, and Stages

An APT campaign is usually described as a multi-stage process rather than as a single attack event. In this paper, we represent such a process using an APT playbook. A playbook contains a sequence of stages, and each stage contains one or more attack actions. An attack action is the smallest unit that can be aligned with a defense observation and later used for scoring.

The action is also the level at which the benchmark preserves evidence: outcomes are recorded per action before being aggregated to stages and playbooks, so that a campaign-level verdict never becomes the only record of how the defense performed.

Stages also differ in how their actions contribute to attacker progress. In some stages, several actions represent alternative ways to achieve the same objective; in others, multiple actions must be completed together; in still others, a group of related actions jointly describes an attacker activity such as discovery or collection. These differences mean that stage-level scoring should not treat every group of actions as a simple average. The exact aggregation rules are defined in Section 4.5; here, the key point is that stage structure is part of the evaluation problem.

3.2 Defense Observations and Evaluation Outcomes

During an evaluation, the components of a deployed defense stack may produce different kinds of evidence, such as prevention records, blocking events, alerts, logs, or analyst-facing detections. We refer to such evidence as defense observations.

A defense observation is not yet a score. It first needs to be aligned with a benchmark action. After alignment, the observation can be normalized into an action-level outcome. In this paper, we use four outcome levels: `PREVENTED`, `BLOCKED`, `DETECTED`, and `MISSED`. These levels describe the observed defense result for a validly exercised action: removed by design, interrupted at runtime, observed but not stopped, or neither stopped nor observed. Whether an action could be validly exercised at all is a separate question of execution validity and is not represented in this four-level vocabulary.

Because these outcomes carry different defensive meaning, the outcome level is preserved on each action before results are aggregated into larger scores.

3.3 From Attack Behavior to Control-Point Attribution

Attack descriptions and defense repair targets are not the same object. An attack technique describes what the attacker did. A reference primary control-point attribution identifies the victim-side control point that the benchmark associates with the action for repair-oriented analysis. For repair-oriented evaluation, the second is essential.

Attacker-side technique labels and victim-side control-point attributions answer different questions. Actions carrying the same technique label may be associated with different control-point attributions depending on their context, so an attack label alone does not determine the repair-oriented reference for an action.

We therefore use the notion of a victim-side control point. A control point is a defensive mechanism, configuration, policy, monitoring capability, or operational process that can be evaluated and improved. For each in-scope action, the benchmark records a reference primary control-point attribution. This attribution does not replace attack-phase information. Instead, it complements it: the control-point attribution identifies which class of victim-side control is associated with the action, while the campaign phase describes where the action occurs in campaign progression.

3.4 Problem Statement

The problem studied in this paper can be stated as follows. Given a set of APT playbooks and action-level defense observations, construct a scoring framework that measures defense capability against multi-stage APT behaviors while keeping the resulting scores interpretable and repair-oriented.

More specifically, the framework should satisfy three requirements.

(1) **Repair-oriented attribution.** Each in-scope attack action should be connected to a victim-side control-point attribution that supports repair-oriented interpretation. Actions that represent external prerequisites rather than ordinary victim-side cases are retained for playbook completeness and represented explicitly as external-prerequisite cases.
(2) **Stage-aware aggregation.** Action-level outcomes should be aggregated according to the role of the action within its stage and the stage within the playbook. The framework should avoid treating all actions as equally valuable or all stages as having the same attacker-progress logic.
(3) **Traceable score reporting.** Aggregated scores should remain explainable after computation. A low score should be traceable to the relevant playbook, stage, action, defense outcome, product category, and victim-side control-point domain.

These requirements motivate the methodology developed in the next section.

4 Methodology

4.1 Overview

This section describes how the proposed method turns APT playbooks and defense observations into scores that remain traceable after aggregation. The central requirement is traceability: a score should not be interpreted only as an aggregate number, but should remain connected to the playbook, stage, attack action, and reference primary control-point attribution that contributed to it.

Fig. 1 gives the overall workflow. The workflow has three parts. First, benchmark construction converts raw playbook actions into scoring-ready records. For attributed benchmark actions, it records a control-point attribution together with a canonical attack phase, expected defense categories, a confidence label, and stage aggregation semantics. These fields form the defense opportunity map used by the scoring engine.

Second, defense scoring aligns observed defense results with the benchmark records. A tested system reports action-level outcomes, which are normalized into four hit levels: `PREVENTED`, `BLOCKED`, `DETECTED`, and `MISSED`. These hit levels capture whether the action was removed by design, interrupted at runtime, only observed, or missed.

Third, the scoring model aggregates the aligned hit values. The aggregation is stage-aware: OR stages, AND stages, and Cluster stages use different rules because they represent different attacker progress conditions. The framework then reports not only an overall score, but also phase-wise scores, control-point domain scores, product-category scores, and action-level attribution details.

4.2 Defensive Weakness Attribution

The first step is to view each attack action from the defender's side. For every in-scope action, we record a reference primary control-point attribution: a single victim-side control point associated with that action for repair-oriented reporting. A control point may be a defensive mechanism, a configuration, a policy, or an operational process that is treated as a distinct concern for evaluation and remediation. The attribution is assigned during benchmark construction, before any defense system is evaluated. It indicates where remediation would apply for that action; it does not record that the control failed in a particular evaluated-system run.

Not every playbook action corresponds to a directly repairable victim-side condition. Pre-compromise reconnaissance, attacker-side infrastructure preparation, and similar external prerequisite activities are retained for playbook completeness and are represented explicitly as external prerequisites, separately from ordinary victim-side cases.

This attribution step is needed because an ATT&CK label describes what the attacker did, but does not by itself determine which victim-side control-point attribution should be associated with the action for repair-oriented analysis. Which control point is the appropriate reference depends on the context of the action and on the victim-side conditions the benchmark represents for it. Actions carrying the same attack label can therefore be associated with different control-point attributions, and the same attribution can be reached from different labels.

We therefore interpret each action by asking a repair-oriented question: if only one control point could be fixed, which one would most directly prevent or disrupt this action? The answer is organized through a structured control-point schema. This schema is not organized as an attack sequence. It separates defensive failures along four design dimensions: security function, architectural location, defensive lifecycle phase, and trust source. These dimensions

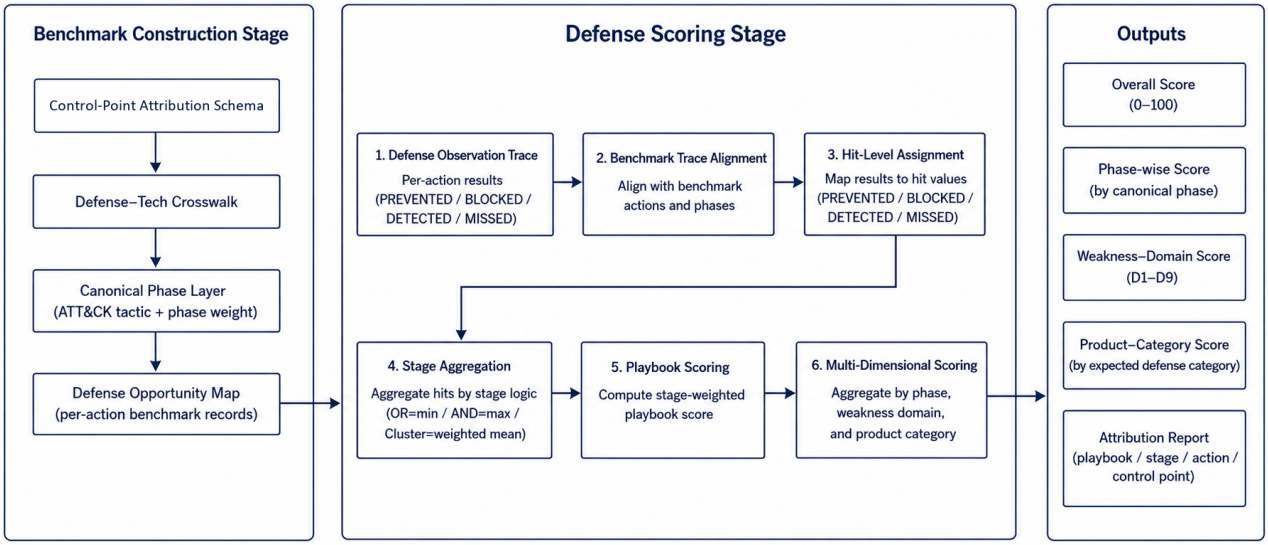

Figure 1: Workflow of the APT defense capability scoring framework.

are not intended to form a strictly orthogonal category system. They are used as practical design constraints for separating control points that lead to different repair actions.

Security function follows the long-standing view that protection mechanisms should be separated by their security roles, such as authentication, authorization, privilege separation, execution control, and data protection [43]. Architectural location captures where the relevant trust boundary or enforcement point sits, which is consistent with zero-trust architecture's shift from static network perimeters toward users, assets, resources, and policy enforcement around them [41]. Defensive lifecycle phase distinguishes controls that prevent an action from succeeding from controls that detect, respond to, or recover from successful actions, aligning with the preventive–reactive distinction in cyber defense modeling [62]. Trust source captures whether the failed control is tied to externally supplied inputs, such as phishing messages, malicious documents, third-party collaboration, or supply-chain artifacts; this is related to attack-surface models that treat entry points, channels, and untrusted data items as key resources through which attackers interact with a system [27].

These dimensions also clarify why the nine domains are not arranged as an attack timeline. The domains describe different ways in which victim-side controls can fail, while the separate canonical phase layer captures when an action occurs in the attack chain. The schema contains nine top-level control-point domains and 97 active leaf-level control points; Table 2 summarizes the top-level domains. The domains indicate where score loss is concentrated, while the leaves provide concrete control-level references for remediation.

Each in-scope action is assigned exactly one primary weakness leaf. This single-label rule keeps later scoring traceable. If one action had multiple primary attributions, the same action could contribute several times, and score loss could not be linked to a clear remediation target. To keep attribution consistent, the schema uses explicit boundary rules and the repair-oriented decision principle described above.

Table 2: Top-level control-point domains.

Domain	Scope
D1	Identity, credential, and trust-root failures
D2	Authorization, isolation, and boundary-control failures
D3	Execution, host, and runtime-protection failures
D4	External exposure and remote-entry failures
D5	Communication and protocol-design failures
D6	Data protection, cryptographic-material, and recovery failures
D7	Observability, detection, and response failures
D8	External trust-input failures
D9	Availability and resilience failures

Some actions still involve more than one relevant weakness. Secondary weakness tags record such supporting conditions, including dual-control dependencies and entry–root-cause splits. These tags help explain attack chains, but they are not counted as primary attribution results and do not replace the primary leaf used for scoring.

After this step, raw attack actions have been converted into attributed benchmark records carrying reference primary control-point attributions. This attributed action set is the basis for canonical phase assignment, defense opportunity mapping, and stage-aware scoring.

4.3 Benchmark Construction

Control-point attribution identifies which victim-side control point the benchmark associates with an action, but it is not enough for scoring by itself. The scoring engine also needs to know when the

action occurs in the attack chain, which defense product categories are expected to address it, and how the action should be aggregated with other actions in the same stage. Benchmark construction adds this information without changing the primary attribution result.

The process has three parts. Attributed actions are first assigned canonical attack phases using ATT&CK. Their control-point leaves are then linked to expected defense product categories. Finally, the enriched records are written into the defense opportunity map, which serves as the direct input to defense scoring.

4.3.1 Canonical Phase Assignment. The attribution domains describe which class of victim-side control is associated with an action; they deliberately do not encode when the action occurs in the attack chain. This temporal information is still needed for scoring and reporting. It allows the framework to distinguish defense performance across attack phases, apply phase-aware weights, and interpret stage-level results in relation to attacker progress. For example, interrupting an early action usually has more defensive value than observing a late action after the attacker has already progressed through several stages. We therefore add a separate canonical phase layer.

For each action, the framework assigns a canonical phase based on its ATT&CK technique. We use ATT&CK Enterprise tactics as the standard phase vocabulary. Some ATT&CK techniques can be associated with multiple tactics. We resolve this ambiguity using a predefined technique-to-phase mapping. Each technique is mapped to a single canonical phase before scoring, and this mapping remains fixed during defense scoring.

Stage-level phases are then derived from the action-level phases, for example by majority voting among the actions in the same playbook stage. This phase assignment is an additional benchmark field; it does not change the control-point attribution described in the previous subsection. The canonical phase field is later used by the scoring model to apply phase weights. The main intuition is that earlier interruption has higher defensive value, while late-stage detection provides less benefit because the attacker has already progressed through much of the playbook.

4.3.2 Defense-Category Mapping. The primary weakness leaf identifies the attributed control point. A scoring benchmark also needs to know which type of defense product should be expected to address that control point. For this purpose, we build a mapping from control-point leaves to defense product categories.

Each fourth-level leaf is mapped to two sets of defense categories. The first set is the primary defense category set. It contains product types that should directly prevent or block the corresponding weakness if deployed and configured correctly. The second set is the compensating defense category set. It contains product types that may detect, mitigate, or partially compensate for the weakness, but are not the most direct repair target.

For example, a brute-force action attributed to weak password and anti-brute-force policy is primarily related to identity and access management, while privileged access management or log monitoring may provide compensating support. Similarly, a C2 communication action attributed to missing egress proxy enforcement may be primarily related to network access control or next-generation firewall capabilities, while SIEM or IDS may provide secondary visibility.

Table 3: Main fields in a defense opportunity record.

Field	Meaning
playbook, stage, action	Identifier of the attack action in the playbook
attack_id	ATT&CK technique or sub-technique identifier
primary_leaf	Reference primary control-point leaf
primary_l1	Top-level control-point domain of the primary leaf
secondary_leaves	Contextual secondary annotations; not scored as primary attributions
canonical_phase	Standard attack phase used for phase-aware scoring
primary_defense	Product categories expected to directly address the weakness
compensating_defense	Product categories that may provide secondary detection or mitigation
confidence	Confidence of the primary attribution
stage_logic	Aggregation semantics used later at the stage level

This mapping allows later scoring to evaluate each product category within its expected coverage set. A product is therefore not judged against all benchmark actions equally, but against the actions that its category is expected to cover.

4.3.3 Defense Opportunity Map. The final output of benchmark construction is the defense opportunity map. Each record in the map corresponds to one attack action and combines the information needed for scoring: the playbook identifier, stage number, action description, ATT&CK technique, primary control-point leaf, top-level domain, canonical phase, expected defense categories, confidence label, and stage aggregation semantics.

The opportunity map keeps the primary attribution unique, but it can also record additional defense opportunities when they are available. In particular, secondary weakness tags can be expanded into additional leaves and defense categories. This preserves the repair-oriented primary attribution while still representing layered defense opportunities.

The opportunity map also records the aggregation semantics used for each playbook stage. These semantics specify how the action-level hit values within a stage should be combined during scoring. For example, a stage may represent alternative attack paths, sequentially required actions, or a loose cluster of related actions. The detailed aggregation rules are defined in the scoring model.

Table 3 summarizes the main fields used in the defense opportunity map.

Through these steps, benchmark construction turns attributed attack actions into structured defense opportunities. The resulting records connect four pieces of information required for scoring: what the attacker did, which control point the benchmark attributes to the action, which defense categories were expected to cover the action, and how the action should contribute to stage-level aggregation.

Table 4: Defense observation results and hit values.

Result	Hit	Meaning
PREVENTED	1.0	The attack path is architecturally removed before the action can occur.
BLOCKED	0.9	The action is attempted but synchronously interrupted by the defense system.
DETECTED	0.5	The action is observed or alerted, but not stopped.
MISSED	0.0	The action is neither prevented, blocked, nor detected.

4.4 Defense Result Alignment

Defense scoring starts by aligning the evaluated system's observations with the benchmark records. Each observation describes how the system responded to a specific playbook action. The alignment step matches the observation to the corresponding defense opportunity record and then converts the observed result into a normalized hit value.

We represent the defense observation trace as a set of action-level records. Each record contains the playbook identifier, the stage number, the action field used in the benchmark, and the observed result. Each defense observation is matched to the action record in the defense opportunity map with the same playbook identifier, stage number, and action field. After alignment, the observed result is associated with the action's control-point attribution, canonical phase, expected defense categories, confidence label, and stage aggregation semantics.

For an in-scope action that has been validly exercised, the absence of a matched defense observation is normalized as MISSED. Under this rule, such an action receives a positive hit value only when the evaluated defense system reports that it was prevented, blocked, or detected. Actions that could not be validly exercised—because the environment does not support them, because the execution attempt was invalid, or because the case is otherwise not assessable—are handled as a question of evaluation scope and are not assigned a defense outcome. Restricting the default in this way keeps the benchmark comparable across different defense systems: every system is evaluated against the same set of in-scope actions, and missing observations within that set are interpreted as lack of demonstrated coverage.

Each matched observation is then converted into one of four hit levels. PREVENTED denotes architectural prevention, where the attack path is removed before the action can be attempted. BLOCKED denotes real-time interruption, where the action is attempted but stopped by the defense system. DETECTED denotes successful observation or alerting without stopping the action itself. MISSED denotes the absence of prevention, blocking, or detection. These levels are mapped to numerical hit values used by the scoring model, as summarized in Table 4.

The distinction between PREVENTED and BLOCKED is important. Prevention means that the attack path is unavailable by design, such as when an egress policy makes an unauthorized destination unreachable. Blocking means that the action is observed and interrupted during execution, such as when an endpoint product terminates a malicious process. Both are stronger than detection-only outcomes, but prevention is assigned a slightly higher hit value because it does not depend on runtime recognition of the specific attack instance.

This alignment step produces a unified action-level scoring input. Each benchmark action is associated with both an expected defense opportunity and an observed defense result.

4.5 Stage-Aware Scoring Model

After defense result alignment, each benchmark action has an action-level hit value and a set of benchmark metadata, including its canonical phase, attribution confidence, and stage aggregation semantics. The scoring model uses these fields to compute defense scores at three levels: action, stage, and playbook.

The main design principle is to separate defense quality from stage importance. The stage score measures how well the defense performed within a stage, while the stage weight determines how much that stage contributes to the playbook-level score. Let a denote an attack action. Its hit value $h(a) \in [0, 1]$ is obtained from the aligned defense result described in the previous subsection.

The action weight $w(a)$ is defined as

$$w(a) = W_{\text{phase}}(\phi(a)) \cdot C_{\text{conf}}(\gamma(a)),$$

where $\phi(a)$ is the canonical phase assigned to action a, W_{phase} is the phase-weight function, $\gamma(a)$ is the attribution confidence label, and C_{conf} is the confidence factor. In our benchmark, the confidence factor reduces the contribution of actions whose attribution is less certain. The implemented factors are 1.0 for high-, 0.8 for medium-, and 0.5 for low-confidence attributions.

For a playbook stage s, let A_s be the set of actions in that stage. The stage score S_s is computed according to the aggregation semantics of the stage. We use three aggregation types: OR, AND, and Cluster.

For an OR stage, the attacker only needs one action in the stage to succeed in order to proceed. Therefore, the defense must block all actions in the stage. The stage score is defined as

$$S_s = \min_{a \in A_s} h(a).$$

This reflects the weakest-link constraint: if any action is missed, the stage is considered poorly defended.

For an AND stage, the attacker needs all actions in the stage to succeed in order to complete the stage objective. Therefore, blocking any one required action can disrupt the stage. The stage score is defined as

$$S_s = \max_{a \in A_s} h(a).$$

This reflects the strongest defensive interruption within the stage.

For a Cluster stage, the actions do not have a strong logical dependency. They are treated as a group of related behaviors rather than as strict alternatives or strict requirements. The stage score is computed as a weighted average:

$$S_s = \frac{\sum_{a \in A_s} h(a) w(a)}{\sum_{a \in A_s} w(a)}.$$

In this case, actions with higher phase importance or higher attribution confidence contribute more to the stage score.

The stage weight is computed separately from the stage score:

$$W_s = \sum_{a \in A_s} w(a).$$

This separation avoids mixing the quality of defense within a stage with the importance of that stage in the overall playbook. In particular, OR and AND stage scores are not weighted internally, because their semantics are determined by the attacker's path structure. The weights are used when aggregating stages into a playbook score.

For a playbook P with stages $S(P)$, the playbook-level score is

$$\text{Score}(P) = \frac{\sum_{s \in S(P)} W_s S_s}{\sum_{s \in S(P)} W_s}.$$

The overall defense capability score is then computed by aggregating the scores over all evaluated playbooks:

$$\text{Score}_{\text{overall}} = \frac{1}{|\mathcal{P}|} \sum_{P \in \mathcal{P}} \text{Score}(P),$$

where $\mathcal{P}$ denotes the set of playbooks in the benchmark.

This formulation keeps the final score traceable: a score loss can be traced back from the overall score to a playbook, then to a stage, and finally to the individual actions and their attributed control points that account for the loss.

4.6 Multi-Dimensional Score Outputs

The overall score is useful, but it is only a starting point. Two defense systems may receive similar aggregate scores for different reasons: one may detect many actions late, while another may cover fewer actions but block them reliably within its intended scope. For this reason, the framework reports scores from multiple perspectives: overall performance, attack phase, control-point domain, defense product category, and action-level attribution.

The phase-wise score groups results by canonical attack phase. This view answers the question: at which attack phases does the defense system lose the most score? Since the scoring model assigns higher value to earlier interception, phase-wise results help distinguish early-stage blocking from late-stage observation. This directly supports stage-aware analysis of APT defense capability.

The control-point domain score groups results by the primary victim-side control-point domain. This view answers a different question: which control-point domain accounts for the most score loss? For example, a low score in an identity-related domain points to weaknesses in credential and trust controls, while a low score in an observability-related domain indicates insufficient detection or response. This decomposition turns an aggregate score into repair-oriented feedback.

The product category score evaluates performance within the expected coverage of each defense product category. For a product category c, let A_c denote the set of actions whose defense opportunity records include c as a primary or compensating defense category. The coverage of c is the fraction of benchmark actions that fall into this expected coverage set:

$$\text{Coverage}(c) = \frac{|A_c|}{|\mathcal{A}|},$$

where $\mathcal{A}$ is the set of benchmark actions.

The hit rate of c is computed only within its coverage set:

$$\text{HitRate}(c) = \frac{\sum_{a \in A_c} h(a)}{|A_c|}.$$

Every action in the coverage set contributes its observed hit value $h(a)$; the main score does not apply a separate numerical credit to primary and compensating matches. The primary/compensating distinction is retained as crosswalk context describing how directly a category relates to the attributed control point.

Reporting both coverage and hit rate is important. Coverage describes how much of the benchmark a product category is expected to address. Hit rate describes how well it performs within that expected scope. A product with narrow coverage but high hit rate has a different defense profile from a product with broad coverage but low hit rate, even if their aggregate scores are similar.

Finally, the framework produces an action-level attribution report. For each score loss, the report retains the playbook, stage, action, ATT&CK technique, canonical phase, primary control-point leaf, defense categories, and observed result. This report provides the trace from aggregate score back to concrete attack actions and their attributed control points. As a result, the output can support both high-level comparison and detailed remediation analysis.

4.7 Prototype Method Validation

The preceding subsections define the attribution schema, benchmark construction process, result alignment procedure, scoring model, and multi-dimensional outputs. Before applying the framework to real defense-system replay experiments, we perform a set of prototype validation experiments on the benchmark and scoring infrastructure. These experiments test whether the proposed methodological components are reproducible, whether the control-point–to–defense mapping is reasonable, and whether the scoring engine behaves correctly under boundary and parameter-variation cases.

First, we assess the reproducibility of the attribution coding scheme through a pilot inter-rater reliability study. A stratified sample of 50 ATT&CK techniques and sub-techniques was independently labeled by two large language model annotators, GPT-4o and Gemini, working from the same coding guide. The pilot was run in the Technique Mapping usage mode, in which the unit of annotation is an ATT&CK technique rather than a playbook action; the separate Incident Chain usage mode used for production playbook annotation was not exercised. Agreement was measured over the coding-guide dimensions: scope decision, top-level domain, leaf-level attribution, confidence label, mapping method, and secondary weakness tag. Mapping method records how an attribution was reached—by explicit rule, inheritance, nearest neighbor, or external prerequisite—and is a different construct from usage mode, which records the unit of annotation. Table 5 summarizes the results.

These figures are preliminary evidence that the coding scheme itself is reproducible: agreement at the top-level domain and leaf levels indicates that the main control-point attribution can be applied consistently by independent annotators working from the guide. Three limits bound this evidence. The pilot predates the current v3.5 schema boundary, so it does not validate leaves that were added or revised afterwards. Both annotators were language models, so the study measures coding-scheme reproducibility rather

Table 5: Prototype validation of defensive weakness attribution.

Field	Agreement	κ	Interpretation
Scope	100.0%	1.000	Almost perfect
L1 domain	96.0%	0.953	Almost perfect
L4 leaf	86.0%	0.856	Almost perfect
Confidence	82.0%	0.679	Substantial
Mapping method	82.0%	0.687	Substantial
Secondary tag	66.0%	0.173	Slight

Table 6: Prototype validation of defense-category mapping.

Validation item	Result
Primary defense internal agreement	97.1% (34 / 35 sampled leaves), $\kappa = 0.485$
Compensating defense internal agreement	45.7% (16 / 35 sampled leaves), $\kappa = -0.094$

than human inter-rater reliability. And the unit of annotation was an ATT&CK technique, so the pilot does not validate the production playbook action annotations or the Incident Chain annotations derived from them. The secondary tag in particular reaches only slight agreement ($\kappa = 0.173$); secondary tags are therefore retained as qualitative contextual information and are not used as primary quantitative validation.

Second, we check the internal consistency of the defense-category mapping used by the Defense–Tech Crosswalk. An independent annotator mapped a stratified sample of 35 of the 97 active leaves to the 29 defense categories without seeing the existing crosswalk. Table 6 summarizes the result. This check covers roughly one third of the active leaves and is not a validation of the crosswalk as a whole.

Within the sampled leaves, the primary defense mapping is the more consistent of the two. Compensating-defense assignments show agreement no better than chance ($\kappa = -0.094$), indicating that compensating coverage is not sharply defined. Compensating mappings are therefore reported as descriptive crosswalk context only; they do not carry a separate numerical weight in the main score.

Third, we check the mathematical behavior of the scoring model using synthetic traces. These traces are not defense experiments; they are boundary checks on the scoring engine. A perfect-defense trace assigns `PREVENTED` to every benchmark action, and a no-defense trace assigns `MISSED` to every action. Table 7 reports the resulting scores.

The perfect and none traces verify the expected upper and lower bounds of the scoring engine: a fully preventing trace scores 100 and a fully missing trace scores 0 under the aggregation rules defined above. Idealized single-category traces, which assign a blocking outcome only within one product category's expected primary coverage, are structural applicability constructions rather than measurements of any deployed product; we therefore do not report them here as current results and do not treat them as evidence about commercial defense performance.

Table 7: Prototype scoring checks with synthetic coverage traces.

Simulation	Configuration	Score
Perfect	All actions are `PREVENTED`	100.00
None	All actions are `MISSED`	0.00

We do not report a quantitative sensitivity analysis at this stage. The scoring model exposes three tunable elements—the phase-weight schedule, the confidence factors, and the stage-level aggregation choice—and their effect on reported scores is examined together with the evaluation results, so that the reported ranges correspond to the population actually scored.

The later evaluation section will use real defense-system observations to test how deployed defenses perform under this methodology.

5 Evaluation

6 Discussion

7 Conclusion

References

[1] Stefan Axelsson. 2000. The Base-Rate Fallacy and the Difficulty of Intrusion Detection. *ACM Transactions on Information and System Security (TISSEC)* 3, 3 (2000), 186–205. doi:10.1145/357830.357849

[2] Tristan Bilot, Baoxiang Jiang, Zefeng Li, Nour El Madhoun, Khaldoun Al Agha, Anis Zouaoui, and Thomas Pasquier. 2025. Sometimes Simpler is Better: A Comprehensive Analysis of State-of-the-Art Provenance-Based Intrusion Detection Systems. In *34th USENIX Security Symposium*. 7193–7212. https://www.usenix.org/conference/usenixsecurity25/presentation/bilot

[3] Shawn A. Butler. 2002. Security Attribute Evaluation Method: A Cost-Benefit Approach. In *ICSE 2002*. 232–240. doi:10.1145/581339.581370

[4] Phuong Cao, Eric C. Badger, Zbigniew T. Kalbarczyk, and Ravishankar K. Iyer. 2016. A Framework for Generation, Replay, and Analysis of Real-World Attack Variants. In *HotSoS 2016 (Symposium and Bootcamp on the Science of Security)*. 28–37. doi:10.1145/2898375.2898392

[5] Martin Casado, Tal Garfinkel, Aditya Akella, Michael J. Freedman, Dan Boneh, Nick McKeown, and Scott Shenker. 2006. SANE: A Protection Architecture for Enterprise Networks. In *Proceedings of the 15th USENIX Security Symposium*.

[6] Zijun Cheng, Qiujian Lv, Jinyuan Liang, Yan Wang, Degang Sun, Thomas Pasquier, and Xueyuan Han. 2024. KAIROS: Practical Intrusion Detection and Investigation using Whole-System Provenance. In *Proceedings of the IEEE Symposium on Security and Privacy*. 3533–3551.

[7] Feng Dong, Shaofei Li, Peng Jiang, Ding Li, Haoyu Wang, Liangyi Huang, Xusheng Xiao, Jiedong Chen, Xiapu Luo, Yao Guo, and Xiangqun Chen. 2023. Are we there yet? An Industrial Viewpoint on Provenance-based Endpoint Detection and Response Tools. In *ACM CCS '23*. 2396–2410. doi:10.1145/3576915.3616580

[8] Adam Doupé, Manuel Egele, Benjamin Caillat, Gianluca Stringhini, Gorkem Yakin, Ali Zand, Ludovico Cavedon, and Giovanni Vigna. 2011. Hit 'Em Where It Hurts: A Live Security Exercise on Cyber Situational Awareness. In *ACSAC 2011*. 51–61. doi:10.1145/2076732.2076740

[9] Simon Yusuf Enoch, Chun Yong Moon, Donghwan Lee, Myung Kil Ahn, and Dong Seong Kim. 2022. A Practical Framework for Cyber Defense Generation, Enforcement and Evaluation. *Computer Networks* 208, Article 108878 (2022). doi:10.1016/j.comnet.2022.108878

[10] Arnau Erola, Ioannis Agrafiotis, Jason R. C. Nurse, Louise Axon, Michael Goldsmith, and Sadie Creese. 2022. A System to Calculate Cyber Value-at-Risk. *Computers & Security* 113 (2022), 102545. doi:10.1016/j.cose.2021.102545

[11] FIRST. 2019. Common Vulnerability Scoring System Version 3.1: Specification Document. https://www.first.org/cvss/v3.1/specification-document. Accessed: 2026-05-26.

[12] Sebastian Garcia, Martin Grill, Jan Stiborek, and Alejandro Zunino. 2014. An Empirical Comparison of Botnet Detection Methods. *Computers & Security* 45 (2014), 100–123. doi:10.1016/j.cose.2014.05.011

[13] Google Cloud Mandiant. 2026. M-Trends 2026: Data, Insights, and Strategies From Mandiant Frontline Investigations. https://cloud.google.com/blog/topics/threat-intelligence/m-trends-2026. Accessed: 2026-05-26.

[14] Lawrence A. Gordon and Martin P. Loeb. 2002. The Economics of Information Security Investment. *ACM Transactions on Information and System Security (TISSEC)* 5, 4 (2002), 438–457. doi:10.1145/581271.581274

[15] Akul Goyal, Gang Wang, and Adam Bates. 2024. R-CAID: Embedding Root Cause Analysis within Provenance-Based Intrusion Detection. In *Proceedings of the IEEE Symposium on Security and Privacy*.

[16] Cormac Herley and Paul C. van Oorschot. 2017. SoK: Science, Security and the Elusive Goal of Security as a Scientific Pursuit. In *Proceedings of the IEEE Symposium on Security and Privacy*.

[17] Michael D. Iannacone and Robert A. Bridges. 2020. Quantifiable & Comparable Evaluations of Cyber Defensive Capabilities: A Survey & Novel, Unified Approach. *Computers & Security* 96, Article 101907 (2020). doi:10.1016/j.cose.2020.101907

[18] IBM Security. 2024. Cost of a Data Breach Report 2024. https://www.ibm.com/think/insights/whats-new-2024-cost-of-a-data-breach-report. Accessed: 2026-05-26.

[19] Muhammad Adil Inam, Yinfang Chen, Akul Goyal, Jason Liu, Jaron Mink, Noor Michael, Sneha Gaur, Adam Bates, and Wajih Ul Hassan. 2023. SoK: History is a Vast Early Warning System: Auditing the Provenance of System Intrusions. In *IEEE Symposium on Security and Privacy*. 2620–2638. doi:10.1109/SP46215.2023.10179405

[20] Baoxiang Jiang, Tristan Bilot, Nour El Madhoun, Khaldoun Al Agha, Anis Zouaoui, Shahrear Iqbal, Xueyuan Han, and Thomas Pasquier. 2025. ORTHRUS: Achieving High Quality of Attribution in Provenance-based Intrusion Detection Systems. In *34th USENIX Security Symposium*. 7173–7192. https://www.usenix.org/conference/usenixsecurity25/presentation/jiang-baoxiang

[21] Peter E. Kaloroumakis and Michael J. Smith. 2021. Toward a Knowledge Graph of Cybersecurity Countermeasures. In *Proceedings of the 2021 Workshop on Cyber Security Experimentation and Test (CSET)*. https://d3fend.mitre.org/resources/D3FEND.pdf

[22] Seungsoo Lee, Jinwoo Kim, and Seungwon Shin. 2017. DELTA: A Security Assessment Framework for Software-Defined Networks. In *Network and Distributed System Security Symposium (NDSS)*. https://www.ndss-symposium.org/ndss2017/ndss-2017-programme/delta-security-assessment-framework-software-defined-networks/

[23] David John Leversage and Eric J. Byres. 2008. Estimating a System's Mean Time-to-Compromise. *IEEE Security & Privacy* 6, 1 (2008), 52–60. doi:10.1109/MSP.2008.9

[24] Vector Guo Li, Matthew Dunn, Paul Pearce, Damon McCoy, Geoffrey M. Voelker, and Stefan Savage. 2019. Reading the Tea Leaves: A Comparative Analysis of Threat Intelligence. In *28th USENIX Security Symposium (USENIX Security 19)*. https://www.usenix.org/conference/usenixsecurity19/presentation/li-vector

[25] Xiaojing Liao, Kan Yuan, XiaoFeng Wang, Zhou Li, Luyi Xing, and Raheem Beyah. 2016. Acing the IoC Game: Toward Automatic Discovery and Analysis of Open-Source Cyber Threat Intelligence. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*. 755–766. doi:10.1145/2976749.2978315

[26] Zhen Liu, Changzhen Hu, Chun Shan, and Zheheng Peng. 2023. DEFIA: Evaluate Defense Effectiveness by Fusing Behavior Information of Cyberattacks. *Information Sciences* 646, Article 119375 (2023). doi:10.1016/j.ins.2023.119375

[27] Pratyusa K. Manadhata and Jeannette M. Wing. 2011. An Attack Surface Metric. *IEEE Transactions on Software Engineering* 37, 3 (2011), 371–386. doi:10.1109/TSE.2010.60

[28] Peter Mell, Karen Scarfone, and Sasha Romanosky. 2006. Common Vulnerability Scoring System. *IEEE Security & Privacy* 4, 6 (2006), 85–89. doi:10.1109/MSP.2006.145

[29] Sadegh M. Milajerdi, Rigel Gjomemo, Birhanu Eshete, R. Sekar, and V. N. Venkatakrishnan. 2019. HOLMES: Real-time APT Detection through Correlation of Suspicious Information Flows. In *Proceedings of the IEEE Symposium on Security and Privacy*.

[30] MITRE. 2026. CALDERA: Automated Adversary Emulation Platform. https://caldera.readthedocs.io/. Accessed: 2026-05-26.

[31] MITRE. 2026. MITRE ATT&CK. https://attack.mitre.org/. Accessed: 2026-05-26.

[32] MITRE. 2026. MITRE D3FEND: A Knowledge Graph of Cybersecurity Countermeasures. https://d3fend.mitre.org/. Accessed: 2026-05-26.

[33] MITRE Center for Threat-Informed Defense. 2022. ATT&CK to NIST SP 800-53 Control Mappings. https://ctid.mitre.org/projects/nist-800-53-control-mappings/. Accessed: 2026-08-02.

[34] MITRE Engenuity. 2026. ATT&CK Evaluations: Methodology Overview. https://evals.mitre.org/methodology-overview/. Accessed: 2026-05-26.

[35] Steven Noel, Sushil Jajodia, Brian O'Berry, and Michael Jacobs. 2003. Efficient Minimum-Cost Network Hardening Via Exploit Dependency Graphs. In *ACSAC 2003*. 86–95. doi:10.1109/CSAC.2003.1254313

[36] Daniel Olszewski, Tyler Tucker, Kevin R. B. Butler, and Patrick Traynor. 2025. SoK: Towards a Unified Approach to Applied Replicability for Computer Security. In *34th USENIX Security Symposium*. 469–488. https://www.usenix.org/conference/usenixsecurity25/presentation/olszewski

[37] Xinming Ou, Wayne F. Boyer, and Miles A. McQueen. 2006. A Scalable Approach to Attack Graph Generation. In *Proceedings of the 13th ACM Conference on Computer and Communications Security*. ACM, 336–345. doi:10.1145/1180405.1180446

[38] Xinming Ou, Sudhakar Govindavajhala, and Andrew W. Appel. 2005. MulVAL: A Logic-based Network Security Analyzer. In *Proceedings of the 14th USENIX Security Symposium*. USENIX Association. https://www.usenix.org/conference/14th-usenix-security-symposium/mulval-logic-based-network-security-analyzer

[39] Alexander V. Outkin, Brandon K. Eames, Meghan A. Galiardi, Sarah Walsh, Eric D. Vugrin, Byron Heersink, Jacob A. Hobbs, and Gregory D. Wyss. 2019. GPLADD: Quantifying Trust in Government and Commercial Systems: A Game-Theoretic Approach. *ACM Transactions on Privacy and Security (TOPS)* 22, 3 (2019). doi:10.1145/3326283

[40] Alexander V. Outkin, Patricia V. Schulz, Timothy Schulz, Thomas D. Tarman, and Ali Pinar. 2022. Defender Policy Evaluation and Resource Allocation With MITRE ATT&CK Evaluations Data. *IEEE Transactions on Dependable and Secure Computing (TDSC)* (2022). doi:10.1109/TDSC.2022.3165624

[41] Scott Rose, Oliver Borchert, Stu Mitchell, and Sean Connelly. 2020. *Zero Trust Architecture*. NIST Special Publication 800-207. National Institute of Standards and Technology. doi:10.6028/NIST.SP.800-207

[42] Martin Rosso, Michele Campobasso, Ganduulga Gankhuyag, and Luca Allodi. 2022. SAIBERSOC: A Methodology and Tool for Experimenting with Security Operation Centers. *ACM Digital Threats: Research and Practice (DTRAP)* 3, 2, Article 14 (2022). doi:10.1145/3491266

[43] Jerome H. Saltzer and Michael D. Schroeder. 1975. The Protection of Information in Computer Systems. *Proc. IEEE* 63, 9 (1975), 1278–1308. doi:10.1109/PROC.1975.9939

[44] Xiangmin Shen, Zhenyuan Li, Graham Burleigh, Lingzhi Wang, and Yan Chen. 2024. Decoding the MITRE Engenuity ATT&CK Enterprise Evaluation: An Analysis of EDR Performance in Real-World Environments. In *ASIA CCS '24*. 96–111. doi:10.1145/3634737.3645012

[45] Yun Shen and Gianluca Stringhini. 2019. ATTACK2VEC: Leveraging Temporal Word Embeddings to Understand the Evolution of Cyberattacks. In *28th USENIX Security Symposium (USENIX Security 19)*. 905–921. https://www.usenix.org/conference/usenixsecurity19/presentation/shen

[46] Oleg Sheyner, Joshua Haines, Somesh Jha, Richard Lippmann, and Jeannette M. Wing. 2002. Automated Generation and Analysis of Attack Graphs. In *Proceedings of the IEEE Symposium on Security and Privacy*. 273–284.

[47] Robin Sommer and Vern Paxson. 2010. Outside the Closed World: On Using Machine Learning for Network Intrusion Detection. In *IEEE Symposium on Security and Privacy*. 305–316. doi:10.1109/SP.2010.25

[48] Joshua Taylor, Kara Zaffarano, Ben Koller, Charlie Bancroft, and Jason Syversen. 2016. Automated Effectiveness Evaluation of Moving Target Defenses: Metrics for Missions and Attacks. In *ACM Workshop on Moving Target Defense (MTD)*. 129–134. doi:10.1145/2995272.2995282

[49] Erik van der Kouwe, Gernot Heiser, Dennis Andriesse, Herbert Bos, and Cristiano Giuffrida. 2019. SoK: Benchmarking Flaws in Systems Security. In *IEEE European Symposium on Security and Privacy (EuroS&P)*. 310–325. doi:10.1109/EuroSP.2019.00031

[50] Vilhelm Verendel. 2009. Quantified Security is a Weak Hypothesis: A Critical Survey of Results and Assumptions. In *New Security Paradigms Workshop (NSPW)*. 37–49. doi:10.1145/1719030.1719036

[51] Verizon Business. 2025. 2025 Data Breach Investigations Report. https://www.verizon.com/business/resources/reports/2025-dbir-data-breach-investigations-report.pdf. Accessed: 2026-05-26.

[52] Apurva Virkud, Muhammad Adil Inam, Andy Riddle, Jason Liu, Gang Wang, and Adam Bates. 2024. How does Endpoint Detection use the MITRE ATT&CK Framework?. In *33rd USENIX Security Symposium*. 3891–3908. https://www.usenix.org/conference/usenixsecurity24/presentation/virkud

[53] Lingyu Wang, Sushil Jajodia, Anoop Singhal, Pengsu Cheng, and Steven Noel. 2014. k-Zero Day Safety: A Network Security Metric for Measuring the Risk of Unknown Vulnerabilities. *IEEE Transactions on Dependable and Secure Computing* 11, 1 (2014), 30–44.

[54] Lingyu Wang, Steven Noel, and Sushil Jajodia. 2006. Minimum-cost network hardening using attack graphs. *Computer Communications* 29, 18 (2006), 3812–3824. doi:10.1016/j.comcom.2006.06.018

[55] Lingyu Wang, Anoop Singhal, and Sushil Jajodia. 2007. Measuring the Overall Security of Network Configurations Using Attack Graphs. In *Data and Applications Security XXI (Lecture Notes in Computer Science, Vol. 4602)*. Springer, 98–112. doi:10.1007/978-3-540-73538-0_9

[56] Joseph M. Werther, Michael A. Zhivich, Timothy R. Leek, and Nickolai Zeldovich. 2011. Experiences in Cyber Security Education: The MIT Lincoln Laboratory Capture-the-Flag Exercise. In *4th Workshop on Cyber Security Experimentation and Test (CSET 11)*. USENIX Association, San Francisco, CA. https://www.usenix.org/conference/cset11/experiences-cyber-security-education-mit-lincoln-laboratory-capture-flag-exercise

[57] Sophia Yoo, Xiaoqi Chen, and Jennifer Rexford. 2024. SmartCookie: Blocking Large-Scale SYN Floods with a Split-Proxy Defense on Programmable Data Planes. In *33rd USENIX Security Symposium (USENIX Security 24)*. 217–234. https://www.usenix.org/conference/usenixsecurity24/presentation/yoo

[58] Lihua Yuan, Hao Chen, Jianning Mai, Chen-Nee Chuah, Zhendong Su, and Prasant Mohapatra. 2006. FIREMAN: A Toolkit for Firewall Modeling and Analysis. In *Proceedings of the IEEE Symposium on Security and Privacy*. 199–213.

[59] Kara Zaffarano, Joshua Taylor, and Samuel N. Hamilton. 2015. A Quantitative Framework for Moving Target Defense Effectiveness Evaluation. In *ACM Workshop on Moving Target Defense (MTD)*. 3–10. doi:10.1145/2808475.2808476

[60] Menghao Zhang, Guanyu Li, Shicheng Wang, Chang Liu, Ang Chen, Hongxin Hu, Guofei Gu, Qi Li, Mingwei Xu, and Jianping Wu. 2020. Poseidon: Mitigating Volumetric DDoS Attacks with Programmable Switches. In *Network and Distributed System Security Symposium (NDSS)*. https://www.ndss-symposium.org/ndss-paper/poseidon-mitigating-volumetric-ddos-attacks-with-programmable-switches/

[61] Mengyuan Zhang, Lingyu Wang, Sushil Jajodia, Anoop Singhal, and Massimiliano Albanese. 2016. Network Diversity: A Security Metric for Evaluating the Resilience of Networks Against Zero-Day Attacks. *IEEE Transactions on Information Forensics and Security* 11, 5 (2016), 1071–1086.

[62] Ren Zheng, Wenlian Lu, and Shouhuai Xu. 2018. Preventive and Reactive Cyber Defense Dynamics Is Globally Stable. *IEEE Transactions on Network Science and Engineering* 5, 2 (2018), 156–170. doi:10.1109/TNSE.2017.2736567

A Appendix

Received 7 October 2025; revised 24 December 2025; accepted 13 January 2026